Msc(DS&AI,CS)
IOT and Application

Teaching Scheme: Credit 01
Theory/Practical : (01 hrs./2 hrs.) / week

Examination Scheme:
Practical (PR): 50 Marks
Oral (OR): 50 Markss

Dr. Minaxi Doorwar
Assistant Professor in E&Tc Department
JSPM's ICOER, Wagholi, Pune

DISTANCE MEASUREMENT USING SR-04 AND ESP32..

The HC-SR04 ultrasonic sensor operates on the principle of **SONAR** (Sound Navigation and Ranging) to measure distance by calculating the "time of flight" of high-frequency sound waves.

Distance Calculation To find the actual distance, use the following formula based on the speed of sound (approximately 340 m/s or 0.034 cm/µs at room temperature):

$$Distance = \frac{Time \times Speed\ of\ Sound}{2}$$

The sensor has four pins for easy connection to a microcontroller or development board:

VCC: Power supply pin, connects to the +5V power source.

Trig (Trigger): Input pin used to initiate distance measurement by providing a 10µs high pulse.

Echo: Output pin that goes high for a duration equal to the time taken for the ultrasonic wave to travel to the object and return.

GND (Ground): Connects to the ground reference of the system

Step-by-Step Operation

Triggering: The microcontroller (like an Arduino) sends a 10µs HIGH pulse to the sensor's Trig pin.

Sonic Burst: Upon receiving this signal, the sensor automatically emits a burst of eight ultrasonic pulses at a frequency of 40 kHz.

Echo Pin High: Immediately after transmitting the burst, the sensor sets its Echo pin to HIGH.

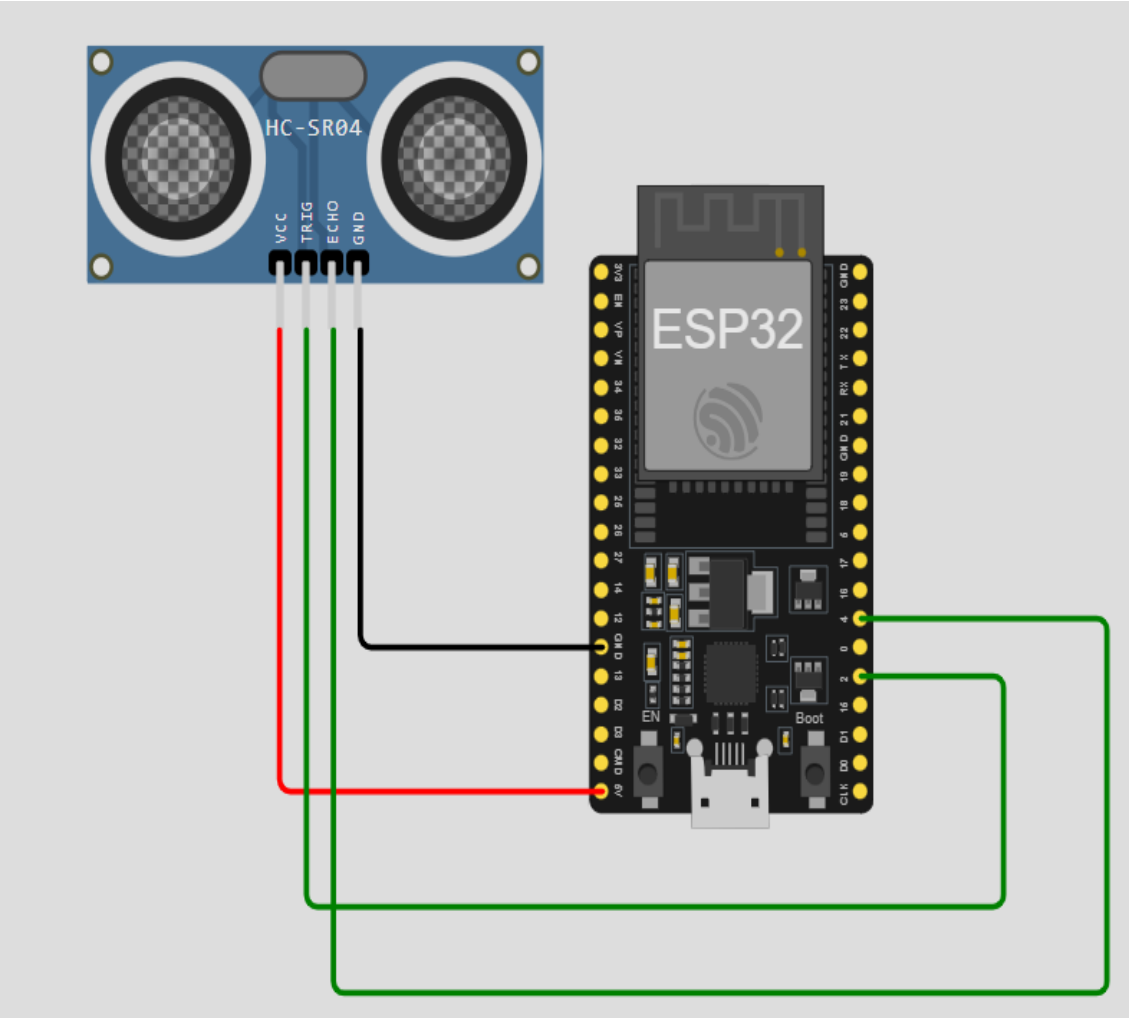
Waiting for Echo: The emitted sound waves travel through the air, hit an object, and bounce back toward the sensor.

Echo Pin Low: When the sensor's receiver detects the returning sound waves, it pulls the Echo pin LOW.

Time Measurement: The microcontroller measures the total time the Echo pin stayed HIGH. This duration represents the total round-trip time for the sound to travel to the object and back.

The pulseIn() function counts how long the Echo pin stays HIGH, which represents the total round-trip travel time of the sound wave

```
const int trigPin = 2;
const int echoPin = 4;
// Variables for duration and distance
int duration;
int distance;

void setup() {
  pinMode(trigPin, OUTPUT); // Sets the trigPin as an Output
  pinMode(echoPin, INPUT);  // Sets the echoPin as an Input
  Serial.begin(195200);       // Starts the serial communication
}
```



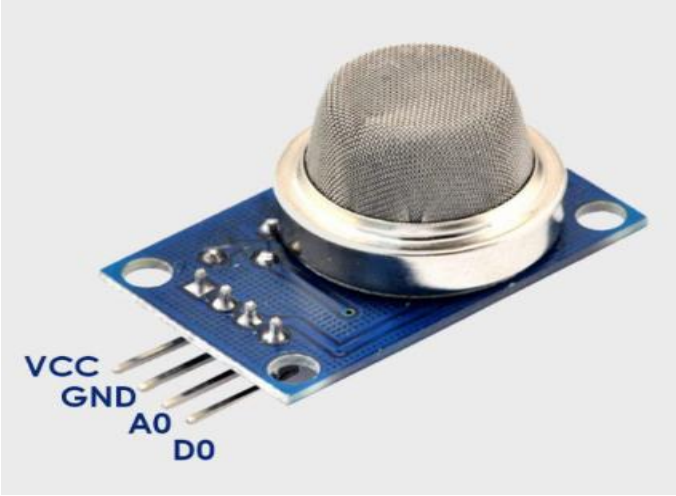
```
void loop() {
  // Clear the trigPin by setting it LOW for 2 microseconds
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  // Trigger the sensor by setting the trigPin HIGH for 10 microseconds
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  // Read the echoPin, returns the sound wave travel time in microseconds
  duration = pulseIn(echoPin, HIGH);
  // Calculate the distance (Speed of sound is 0.034 cm/us)
  distance = duration * 0.034 / 2;
```

```
  // Print the distance on the Serial Monitor
   Serial.print("Duration : ");
  Serial.print(duration);
  Serial.println(" microsecond");
  Serial.print("Distance: ");
  Serial.print(distance);
  Serial.println(" cm");

  delay(500); // Wait half a second before next measurement
}
```

GAS DETECTION USING MQ135/MQ2 AND ESP32.

The MQ-2 sensor is a versatile gas sensor capable of detecting a wide range of gases including alcohol, carbon monoxide, hydrogen, isobutene, liquefied petroleum gas, methane, propane, and smoke. It is popular among beginners due to its low cost and easy-to-use features.

The MQ-2 sensor works on the principle of resistance changes in the presence of different gases. When the target gas comes in contact with the heated MOS (Metal Oxide Semiconductor) material, it undergoes oxidation or reduction reactions that change the resistance of the MOS material.

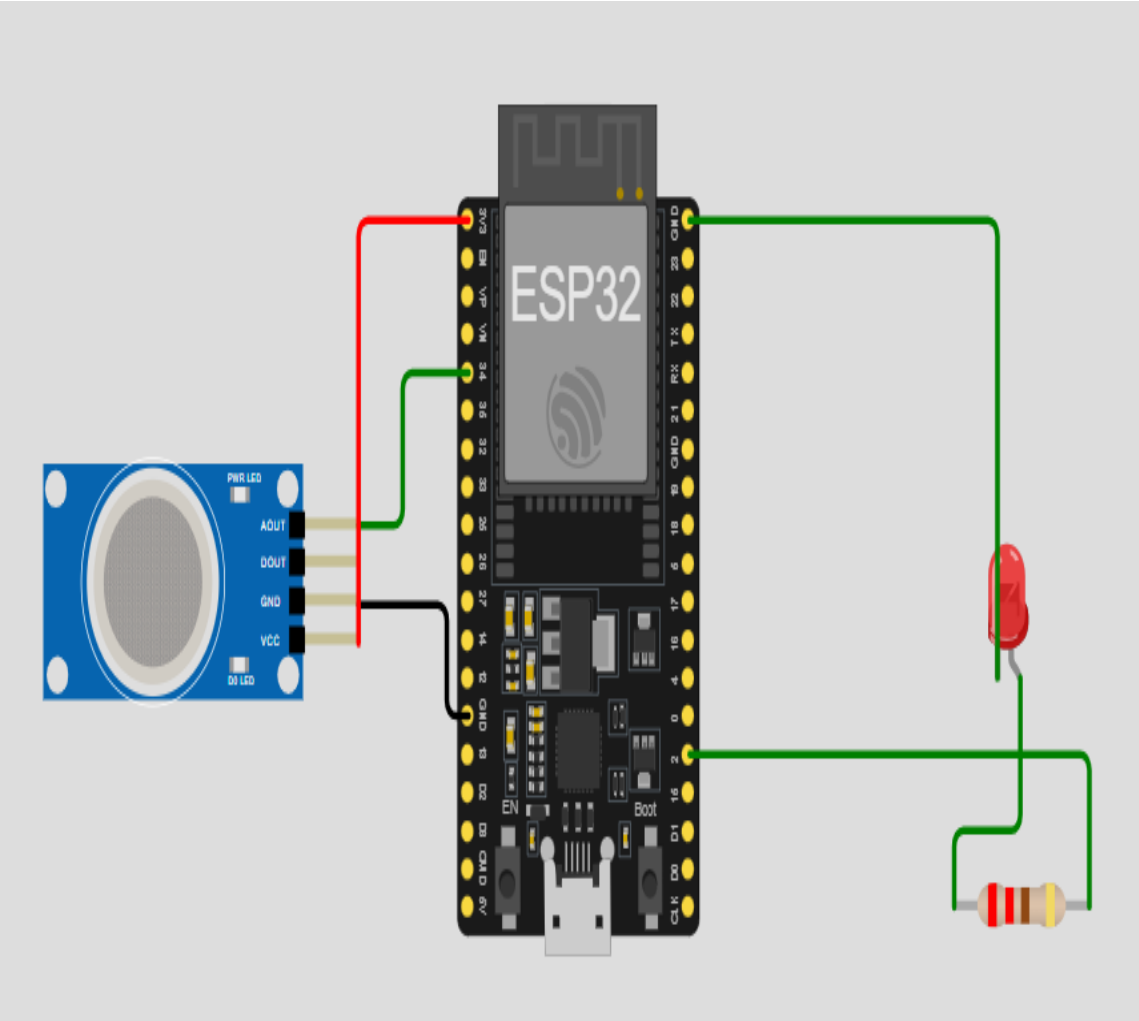


When the concentration of gas in the air exceeds a certain threshold value, the resistance of the sensor changes.The core of the sensor is a ceramic tube coated with a thin layer of SnO_2. In clean air, SnO_2 has a high electrical resistance. This is because oxygen molecules from the atmosphere are adsorbed onto the surface of the SnO_2 layer and "trap" free electrons, creating a potential barrier that prevents current flow. In a metal mesh cover, a small heating filament is present.

It is essential because the chemical reactions required for detection only occur efficiently at high temperatures (typically around **300°C**). This is why the sensor needs a **pre-heating period** and becomes physically hot to the touch during operation.The gas molecules react with the adsorbed oxygen ions.This reaction releases the trapped electrons back into the SnO_2 layer.As free electrons increase, the electrical resistance of the sensor drops.The sensor module uses a simple **voltage divider** circuit with a load resistor RL.

- **Low Gas Concentration:** High sensor resistance = **Low output voltage**.
- **High Gas Concentration:** Low sensor resistance = **High output voltage**.

```
void setup() {
  Serial.begin(115200);
  pinMode(2, OUTPUT); // Internal LED or Buzzer
  Serial.println("Warming up sensor...");
  delay(10000); // Simulated warm-up
}
void loop() {
  float voltage = raw * (3.3 / 4095.0);
  int gasvalue = analogRead(34);
    Serial.print("Gas Level: ");
  Serial.println(gasvalue);
```



```
// LOGIC: Trigger if value is ABOVE the threshold
if (gasvalue > 1000) {
  digitalWrite(2, HIGH);
  Serial.println("!!! DANGER: Gas Leak Detected !!!");
}
else {
  digitalWrite(2, LOW);
  Serial.println("Status: Environment safe");
}

}
```

SERVO MOTOR (SG-90) CONTROL USING ESP32

A servo like SG90 moves based on **pulse width**, not directly on duty cycle.

Standard PWM frequency: **50 Hz** → period = 20 ms

- **PWM Frequency:** 50 Hz (standard for servos).
- **Total Period (T):** $\frac{1}{50\text{ Hz}} = 0.02$ seconds, which is **20,000 microseconds (µs)**.
- **12-bit Resolution:** This means the 20,000 µs period is divided into 2^{12} equal parts, which is **4,096 steps**.

The Calculation:

$$\text{Step Size} = \frac{\text{Total Period}}{\text{Total Steps}} = \frac{20,000\ \mu s}{4,096} \approx \mathbf{4.88\ \mu s}$$

Internal Components

A standard hobby servo contains four essential parts working in unison:

DC Motor: The actual drive unit that provides rotation.

Gearbox: A set of gears (plastic in SG-90) that reduces the motor's high speed to increase torque (lifting power).

Potentiometer (Position Sensor): A variable resistor connected to the output shaft that provides a voltage signal representing the shaft's actual position.

Control Circuit: The "brain" that receives commands and manages the feedback loop.

Pulse width range (usually for SG90):

- **500 µs → 0°**
- **1500 µs → 90°**
- **2400 µs → 180°**

Because a servo responds to **pulse width**, a mathematical **mapping** is used:

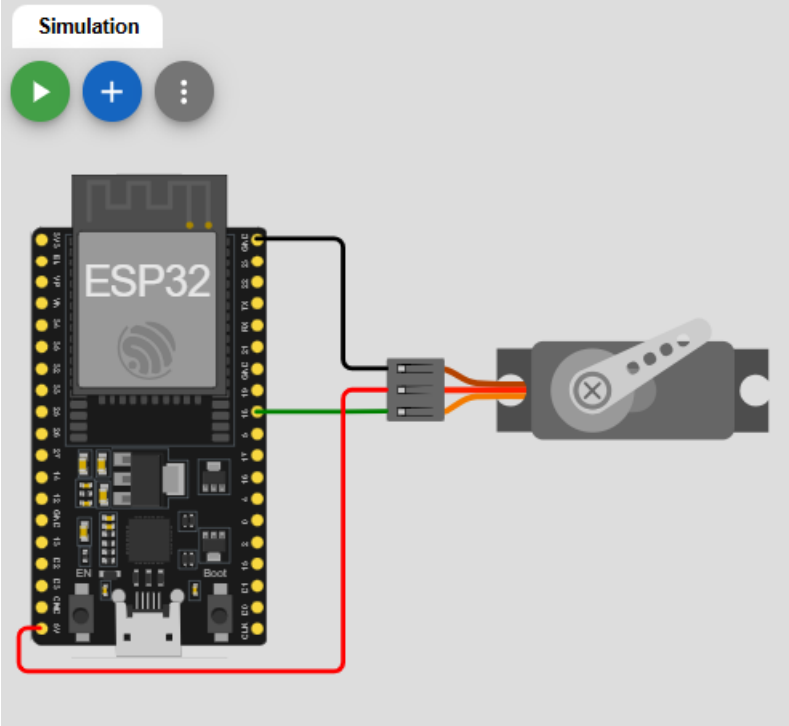
$$\text{Pulse } Width\ (\mu s) = MinPulse + \frac{Angle}{180} \times (MaxPulse - MinPulse)$$

Where: MinPulse = 500 µs → 0°

MaxPulse = 2400 µs → 180°

Angle = 0°–180°

```
#include <ESP32Servo.h>

// Define the GPIO pin the servo signal wire is connected to

static const int servoPin = 18;

Servo myservo;  // Create a servo object to control the servo

void setup() {

  Serial.begin(115200); // Start serial communication

  myservo.setPeriodHertz(50); // Set the PWM frequency to 50Hz (standard
for hobby servos)

  // Attach the servo object to the pin and define min/max pulse width in
microseconds (optional, use defaults for SG90)

  myservo.attach(servoPin, 500, 2400); // 500us for 0 degrees, 2400us for 180
degrees

}
```



```
void loop() {
  // Sweep from 0 to 180 degrees
  for (int pos = 0; pos <= 180; pos += 1) {
    myservo.write(pos); // Tell servo to go to a specific angle
    delay(15);          // Delay for smooth movement
  }
  // Sweep from 180 to 0 degrees
  for (int pos = 180; pos >= 0; pos -= 1) {
    myservo.write(pos); // Tell servo to go to a specific angle
    delay(15);          // Delay for smooth movement
  }
}
```

```
#include <ESP32Servo.h>  // ESP32-compatible servo library
Servo myServo;          // Create servo object
const int potPin = 25;   // Analog input pin for potentiometer
const int servoPin = 18; // PWM output pin to servo

// Servo pulse limits for SG90
const int minPulse = 500;   // Minimum pulse width in microseconds (0°)
const int maxPulse = 2400;  // Maximum pulse width in microseconds (180°)

void setup() {
  myServo.attach(servoPin, minPulse, maxPulse);  // Attach servo with pulse limits
}

void loop() {
  int potValue = analogRead(potPin);            // Read potentiometer (0-4095)
  int angle = map(potValue, 0, 4095, 0, 180);    // Map potentiometer to servo angle
  myServo.write(angle);                         // Move servo to mapped angle
  delay(15);                                // Small delay for smooth motion

}
```